

Provable defenses against adversarial examples via the convex outer adversarial polytope

J. Zico Kolter, Eric Wong
School of Computer Science
Carnegie Mellon University
Pittsburgh PA, 15213, USA
{zkolter,ericwong}@cs.cmu.edu

November 16, 2021

Abstract

We propose a method to learn deep ReLU-based classifiers that are provably robust against norm-bounded adversarial perturbations (on the training data; for previously unseen examples, the approach will be guaranteed to detect all adversarial examples, though it may flag some non-adversarial examples as well). The basic idea of the approach is to consider a *convex outer approximation* of the set of activations reachable through a norm-bounded perturbation, and we develop a robust optimization procedure that minimizes the worst case loss over this outer region (via a linear program). Crucially, we show that the dual problem to this linear program can be represented itself as a deep network similar to the backpropagation network, leading to very efficient optimization approaches that produce guaranteed bounds on the robust loss. The end result is that by executing a few more forward and backward passes through a slightly modified version of the original network (though possibly with much larger batch sizes), we can learn a classifier that is provably robust to *any* norm-bounded adversarial attack. We illustrate the approach on a toy 2D robust classification task, and on a simple convolutional architecture applied to MNIST, where we produce a classifier that provably has less than 8.4% test error for any adversarial attack with bounded ℓ_∞ norm less than $\epsilon = 0.1$. This represents the largest verified network that we are aware of, and we discuss future challenges in scaling the approach to much larger domains.

1 Introduction

Recent work in deep learning has demonstrated the prevalence of *adversarial examples* [Goodfellow et al., 2015], data points fed to a machine learning algorithm which are visually indistinguishable from “normal” examples, but which are specifically tuned so as to fool or somehow mislead the machine learning system. Recent history in adversarial classification has followed something of a virtual “arms race”: practitioners alternatively design new ways of “hardening” classifiers against existing attacks, and then a new class of attacks is developed that can penetrate this defense. Distillation [Papernot et al., 2016] is effective at preventing adversarial examples until it is not [Carlini and Wagner, 2017]. There is no need to worry about adversarial examples under “realistic” settings of rotation and scaling [Lu et al., 2017] until there is [Athalye and Sutskever, 2017]. Neither does the fact that the adversary lacks full knowledge of the model appear to be a problem: “black-box” attacks are also extremely effective [Papernot et al., 2017]. Given the potentially high-stakes nature of many machine learning systems, we feel this situation is untenable: the “cost” of having

a classifier be fooled just once is potentially extremely high, and so the attackers are the de-facto “winners” of this current game.

Rather, we feel that the *only* method to truly harden classifiers against adversarial attacks is to design classifiers that are guaranteed to be robust to adversarial perturbations, even if the attacker is given full knowledge of the classifier. Anything less amounts to an attempt of “security through obscurity”, and will ultimately prove unable to provide a robust classifier. However, designing classifiers that are truly robust to adversarial examples is an extremely challenging task (and potentially even impossible, in many scenarios). However, unlike the current trend of defense mechanisms, which seems to rest largely upon ad-hoc strategies or particular forms of attack, we think that preventing against *any* attack is a crucial property for future machine learning systems.

In this proposed work, we aim to take a step in this direction by considering more formally the actual polyhedral geometry of deep networks. We show (similar to previous work, which we will mention shortly), that we can characterize the set of *all* adversarial examples for a ReLU-based deep network through a (highly non-convex) polytope characterized by an integer program. However, integer programming approaches do not scale to anywhere near the complexity of modern deep learning architectures, and so instead we focus our attention to *outer bounds* of this “adversarial polytope”: convex outer approximations to the adversarial region which *are* possible to define and reason about efficiently. Because these convex approximations are strictly *outer* approximations (i.e., they contain the true adversarial polytope), if we are able to design classifiers with the desired properties over these convex approximations, these same properties are guaranteed to hold over the true adversarial region. Of course, “efficiently” in the language of convex optimization still may mean polynomial in the number of hidden units of network, a situation which is not practically efficient in the deep learning setting.

Thus, the main technical thrust of this work is to develop efficient methods that can optimize over these convex outer approximations while having cost that is “similar” (at least to a couple orders of magnitude) to that of existing deep learning algorithms. The key idea is to consider the dual problem of optimizing over the convex outer adversarial polytope, which provides a guaranteed lower bound on the optimization problem of interest. We show that the dual problem can in fact be characterized exactly via a “dual network” similar to the backpropagation network of the original classifier, but with additional free parameters; because the dual network provides a guaranteed bound on the original convex problem for *any* feasible solution, even if we don’t solve the dual problem exactly, it still provides us with a provable bound. Thus, although our approach is based upon the theory of linear programming and convex optimization, the resulting algorithm just requires passes through an ordinary deep network. The end result is that *with a few additional forward and backward passes through the network (though potentially with much larger batch sizes) we can train a classifier that is guaranteed to be robust against norm-bounded adversarial perturbations.*

We illustrate the proposed method on a few simple problems: a toy 2D data set used to illustrate the basic concepts, and a ConvNet classifier applied to MNIST. In the latter case, we show that we can learn a classifier that provably attains 91% test accuracy against *all* ℓ_∞ -bounded adversarial attacks with radius $\epsilon = 0.1$. Although these are small-scale problems, and the method is admittedly much more costly than traditional learning (though not combinatorially so), to the best of our knowledge this represents the best *provable* accuracy against robust attacks for an MNIST classifier. We discuss the challenges and strategies for scaling these techniques to larger settings. All code for the examples included in the paper is available at http://github.com/locuslab/convex_adversarial.

1.1 Related work

Our work relates most closely to two recent trends in adversarial examples. The first (which is a topic that is broader than merely adversarial examples), is the work in the verification of neural networks [Huang et al., 2017, Katz et al., 2017, Ehlers, 2017]. There are several different approaches to this general task, but the general idea behind this line of work is to formally verify some property of the network, such as robustness to an adversarial perturbation, or that certain bounded inputs will lead to bounded outputs. The methods for solving these verification problems also differ, but one of the primary tools commonly used is Satisfiability Modulo Theories (SMT) solvers. These essentially break the non-linear nature of the ReLU activation into a logical clauses, and then use satisfiability-type solvers to efficiently reason about which clauses may be true or not, and which imply others. Katz et al. [2017] recently developed the Reluplex solver for efficiently solving relatively large-scale problems of this type. The PLANET solver [Ehlers, 2017] uses a similar high level idea, but also combines a satisfiability solver with a convex approximation of the ReLU nonlinearities (at the level of a single ReLU unit, it is the same outer bound as used in this work, though the nature of our full outer bound is different). Similar techniques, though in this case based upon mixed integer programming, have been used to analyze the reachability of inputs [Lomuscio and Maganti, 2017]. SMT methods have also been used to find “ground-truth” adversarial examples Carlini et al. [2017], the best possible adversarial example within a bounded region, or the smallest possible region.

The present work takes a great deal of inspiration from these formal verification methods, and the end goal is largely the same: to develop networks that are guaranteed to have certain properties. However, as evidenced by this past work, arbitrary deep networks are extremely difficult to verify (indeed, as evidenced by the fact that naively-trained networks typically *are* susceptible to adversarial examples). Instead, this work takes an alternative approach: learning deep classifiers that by their nature are (relatively) *easy* to verify, via convex instead of combinatorial problems.

Second, our work relates to the general theme of integrating robust optimization into deep learning. Robust optimization [Ben-Tal et al., 2009] is a very well-studied discipline in convex optimization, and generally deals with optimization problems where the objectives or constraints are not fully known, but only known to lie within some uncertainty set. Of particular relevance to this work is the general technique of using duality theory to replace min-max objectives or constraints with a single minimization; we will also use this approach to develop an efficient solution method for our robust optimization problem. Indeed, robust optimization techniques have been used in the context of linear machine learning models [Xu et al., 2009] to create classifiers that are robust to perturbations of the input. And this connection was addressed in the original adversarial examples paper Goodfellow et al. [2015], where it was noted (the fact is well-known in robust optimization, and we merely mean that the original paper pointed out this connection) that for linear models, robustness to adversarial examples can be achieved via an ℓ_1 norm penalty on the weights within the loss function. However, as also noted by the paper, since deep networks are decidedly non-convex, much of the theory of robust optimization is inapplicable in the deep learning setting, and so approximation methods such as the fast gradient sign method are used instead. More recently, Madry et al. [2017] revisited this connection to robust optimization, and noted that simply solving the (non-convex) min-max formulation of the robust optimization problem works very well in practice to find and then optimize against adversarial examples. Unfortunately, because the method is still heuristically solving a non-convex problem, it is difficult to see how it could be modified to produce true provable guarantees.

This work can be seen as taking the next step in this connection between adversarial examples and robust optimization. Because we consider a convex relaxation of the adversarial polytope,

we can incorporate the theory from convex robust optimization and provide *provable* bounds on the potential adversarial error and loss of a classifier. But we rely on the specific form of the deep network to derive an extremely efficient method for finding feasible dual solutions of the optimization problem in question without relying on any traditional optimization solver.

Finally, our work also relates closely to a paper that was released a few days prior Anonymous [2018]. This paper also considers provable bounds on the possible adversarial loss of a classifier, and uses a convex upper bound (in their case, a semidefinite program), along with the dual problem, to upper bound the robust loss. However, the actual form and apparent function of the bounds differ completely, as their approach uses a semidefinite program for the specific case of one hidden layer MLPs. Additionally, the relaxation they propose seems limited to one hidden layer networks, requires a more specialized optimization procedure that includes computing and minimizing a maximum eigenvector via a Lanczos iteration, and also seems to lead to worse networks: their result highlights a guaranteed bound of 35% error on MNIST, whereas our method achieves a provable 8.3% error bound.¹

2 The adversarial polytope

We begin by considering a characterization of the exact adversarial polytope of a deep network; a very similar approach is used in Lomuscio and Maganti [2017], though we include it here for completeness, as the adversarial polytope plays a foundational role in our subsequent main contribution in the following section.

To fix notation, we consider a k layer feedforward ReLU-based neural network, $f_\theta : \mathbb{R}^{|x|} \rightarrow \mathbb{R}^{|y|}$, where the output of the network is given by the pre-softmax final-layer activations (so that they would correspond to the logits of a multi-class classification architecture). The network structure is given by the equations

$$\begin{aligned}\hat{z}_{i+1} &= W_i z_i + b_i, \quad i = 1, \dots, k-1 \\ z_i &= \max\{\hat{z}_i, 0\}, \quad i = 2, \dots, k-2 \\ z_1 &= x \\ f_\theta(x) &\equiv \hat{z}_k\end{aligned}\tag{1}$$

where we will use $\theta = \{W_1, \dots, W_{k-1}, b_1, \dots, b_{k-1}\}$ to denote the set of all parameters of the network. Although we write the linear terms in the network as matrix operations $W_i z_i + b_i$, we note that W_i could be any linear operator such as a convolution (and we will indeed consider convolutions in our experimental results). We also are assuming that the network here only has ReLU non-linearities. The method we propose can potentially be extended to other polytopic activations such as max-pooling or leaky ReLUs, but we leave this for future work.² We also note that we can integrate more complex networks that have passthrough layers or more complex structure, but again we use the simple feedforward network for simplicity of notation throughout the paper.

Now we consider a set of allowable perturbations $x + \Delta$ where $\|\Delta\|_\infty \leq \epsilon$ (we use the ℓ_∞ norm to bound adversarial examples, as this is one of the most common settings in practice, though other

¹This assessment represents our best understanding of this work at the moment, though the paper is under anonymous review, and thus we not been able to discuss with the authors. We welcome any feedback on this point and will update our notes here as needed.

²We can also potentially integrate other non-piecewise-linear activations such as sigmoids, though this requires an approximation if we do want to use polyhedral methods, so we focus here on piecewise-linear activations.

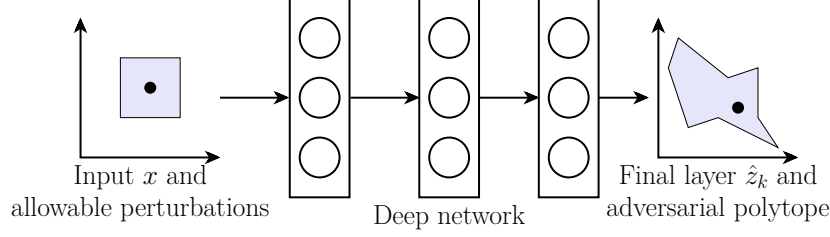


Figure 1: Illustration of the adversarial polytope (i.e., the “reachable” region for a given allowable perturbation).

norms are possible as well). The *adversarial polytope* is defined as *the set of all possible z_k terms achievable under this perturbation*

$$\mathcal{Z}_\epsilon(x) = \{\hat{z}_k | \hat{z}_k = f(x + \Delta), \|\Delta\|_\infty \leq \epsilon\} \quad (2)$$

The conceptual idea of the adversarial polytope is illustrated in Figure 1. This set provides us with all the information we need about adversarial examples (at least those defined by the simple perturbations defined above, which already encompass a powerful set). We will discuss this in more detail shortly, but optimizing over the adversarial polytope can tell us, for instance, whether there is any example within ϵ of x that can change the class of the example. It also forms the basis for developing a robust optimization approach that guarantees that no adversarial examples (at least on the training data) are possible.

Although it may not be apparent, $\mathcal{Z}_\epsilon(x)$ is a connected polytope, a property which follows from the fact that the entire function is composed of linear operations and the non-linear ReLU operator, which is still defined by a (non-convex) polytope: for $y = \text{ReLU}(x)$, the ReLU is defined by the set $(y = 0 \wedge x \leq 0) \vee (y = x \wedge x > 0)$. Indeed, we can express the true adversarial polytope in terms of the *bilinear integer constraints*

$$\begin{aligned} \hat{z}_{i+1} &= W_i \hat{z}_i + b_i, \quad i = 1, \dots, k-1 \\ \hat{z}_i &= \hat{z}_i \cdot y_i, \quad i = 2, \dots, k-1 \\ \hat{z}_1 &\leq x + \epsilon \\ \hat{z}_1 &\geq x - \epsilon \\ y_i &\in \{0, 1\}^{|z_i|} \end{aligned} \quad (3)$$

where y_i is a set of binary integer variables that encode “which side” of the ReLU the pre-ReLU activations $\hat{z}_i$ lies on. Although this is not in a form that can be easily handled by existing IP solvers because of the bilinear term $\hat{z}_i \cdot y_i$, it can be put into an allowable form using a standard trick from integer programming known as *linearization*. Specifically, supposing that we have some lower and upper bounds ℓ_i, u_i on the pre-ReLU activations $\hat{z}_i$ (for the integer programming formulations, these bounds can be very loose, so can be derived for instance from Lipschitz bounds of the network), then the above set of constraints is equivalent to the mixed integer constraints (we don’t cover this

transformation in detail here, but it is a very common pattern in the IP literature)

$$\begin{aligned}
\hat{z}_{i+1} &= W_i z_i + b_i, \quad i = 1, \dots, k-1 \\
\hat{z}_i &\leq u_i \cdot y_i, \quad i = 2, \dots, k-1 \\
z_i &\geq \hat{z}_i, \quad i = 2, \dots, k-1 \\
z_i &\geq 0, \quad i = 2, \dots, k-1 \\
z_i &\leq \hat{z}_i - (1 - y_i) \cdot \ell_i \\
z_1 &\leq x + \epsilon \\
z_1 &\geq x - \epsilon \\
y_i &\in \{0, 1\}^{|z_i|}.
\end{aligned} \tag{4}$$

It is simple to check that if $y_{i,j} = 0$, this implies the corresponding $z_{i,j} = 0$ (the constraints simplify to $z_{i,j} \geq 0$ and $z_{i,j} \leq 0$); similarly, for $y_{i,j} = 1$, the constraints imply that $z_{i,j} = \hat{z}_{i,j}$. This form, which now can be fed to off-the-shelf solvers, lets us reason about the exact nature of adversarial examples for a classifier.

2.1 Queries within the adversarial polytope

The adversarial polytope allows us to formalize the notion of finding or preventing adversarial examples in a classifier. Considering some datapoint/label pair (x, y) , as well as a *target class* y_{targ} (the class that we are going to attempt to fool the classifier into producing), we can determine whether or not there exists an adversarial example (within ϵ ℓ_∞ norm of x) that makes the label y_{targ} *more likely* than y by solving the optimization problem

$$\text{minimize } (z_k)_y - (z_k)_{y_{\text{targ}}}, \quad \text{subject to } z_k \in \mathcal{Z}_\epsilon(x). \tag{5}$$

If the optimal objective of this optimization problem is less than 0, then there *does* exist an adversarial example that makes the label y_{targ} more likely than y ; this follows immediately from the nature of the optimization problem: if the objective is negative, then the value of the y_{targ} activation is greater than the y activation, which guarantees the desired property. This optimization problem is a mixed integer linear program, and (using the polytope formulation provided in (4)) can be solved by a number of off-the-shelf integer programming solvers. Other questions can be answered as well: for instance if we want to find the minimum-perturbation adversarial example possible, then we can solve the MILP

$$\text{minimize } \epsilon, \quad \text{subject to } (z_k)_y \leq (z_k)_{y_{\text{targ}}}, \quad z_k \in \mathcal{Z}_\epsilon(x). \tag{6}$$

Besides creating adversarial examples, the adversarial polytope can also be used to *test* the potential adversarial nature of unknown examples. If we are provided an example $x \in \mathbb{R}^n$ as input to a classifier, a natural question to ask is: is this potentially an adversarial example, e.g., an example that has been crafted to fool our classifier. While it is difficult to answer this question in general, in some cases it *is* possible to guarantee that a particular example is *not* adversarial, at least under the set of allowable perturbations. Specifically, if we let $y_{\text{pred}} = \text{argmax}_i f_i(x)$ be the predicted class of the example, then we can check (using the approach above) whether there are adversarial examples for all $y_{\text{targ}} \neq y_{\text{pred}}$. If no such example exists, then *we know that the network classifies the sample x as being the same class for all ϵ perturbations*, i.e., it could not be an instance of a “good” example (which would be classified correctly) perturbed by ϵ to be classified incorrectly. This allows us a method for guaranteeing that an example is not adversarial, even if we cannot

do the converse, guarantee an example that does cross classes within the adversarial polytope is actually adversarial. However, because we know from experience that most deep classifiers *are* easy to fool, it is likely that a typically-trained network will flag most inputs as being potentially adversarial; thus, as we will address next, we need additional methods for training classifiers such that they are likely to produce classifiers not sensitive to adversarial perturbations.

2.2 Robust optimization and learning robust classifier

Finally, we discuss how we can use the adversarial polytope to develop “hardened” classifiers (more) robust to adversarial examples, based upon approaches in robust optimization. Robust optimization in the context of linear classifiers has been used for some time within machine learning (e.g Xu et al. [2009]). The basic idea is that, given a training set of x_i, y_i pairs, instead of simply minimizing the loss at these data points, we minimize the loss at the *worst* location (i.e. that with the highest loss) in an ϵ ball around each x_i . Assuming a linear classifier $h_\theta(x) = \theta^T x$ (and binary classification, for simplicity), this results in the optimization problem

$$\underset{\theta}{\text{minimize}} \quad \sum_{i=1}^N \max_{\|\Delta\| \leq \epsilon} \ell(\theta^T(x + \Delta) \cdot y_i) \quad (7)$$

Using the the fact that

$$\max_{\|\Delta\| \leq \epsilon} \theta^T \Delta = \epsilon \|\theta\|_* \quad (8)$$

where $\|\cdot\|_*$ denotes the dual norm, the robust optimization problem can be simplified as

$$\underset{\theta}{\text{minimize}} \quad \sum_{i=1}^N \max_{\|\Delta\| \leq \epsilon} \ell(\theta^T x \cdot y_i + \|\theta\|_*). \quad (9)$$

As mentioned above, recent work has highlighted the connection between adversarial classification and robust optimization Madry et al. [2017], in particular attempting to solve the non-convex analogue of the native min-max problem (7) by (projected) gradient descent over θ and Δ ; this connection to robust optimization was also discussed in the original adversarial example paper Goodfellow et al. [2015], with the fast gradient sign method reducing to the same solution of this min-max problem for a linear classifier.

To consider how to use the adversarial polytope to more exactly formulate the robust optimization problem, we consider training a deep classifier with a multiclass hinge loss; other losses such as the more common cross-entropy loss can be easily approximated, but the connection is more obvious with this loss. Optimizing such a deep network is given by the optimization problem

$$\underset{\theta}{\text{minimize}} \quad \sum_{i=1}^N [1 + \max_{y \neq y_i} f_y(x) - f_{y_i}(x)]_+. \quad (10)$$

This loss is zero if the true class output $f_{y_i}(x)$ is at least 1 larger than any alternative class output $f_y(x)$ for $y \neq y_i$, and scales linearly with the largest activation other than the true class otherwise. Because the term $f_y(x) - f_{y_i}(x)$ is exactly the objective that we considered previously, we can encode the min-max formulation of robust optimization by considering $|y| - 1$ different

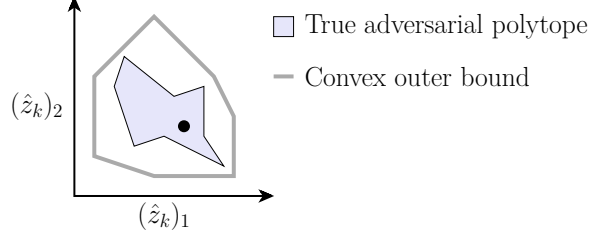


Figure 2: Simple conceptual illustration of the (non-convex) adversarial polytope, and an outer convex bound.

classification problems (one for each alternative label $y \neq y_i$) over the adversarial polytope

$$\begin{aligned}
 & \underset{\theta}{\text{minimize}} \sum_{i=1}^N \max_{\|\Delta\|_{\infty} \leq \epsilon} [1 + \max_{y \neq y_i} f_y(x + \Delta) - f_{y_i}(x + \Delta)]_+ \\
 & \equiv \underset{\theta}{\text{minimize}} \sum_{i=1}^N [1 + \max_{y \neq y_i} \max_{\hat{z} \in \mathcal{Z}_{\epsilon}} (x_i)((\hat{z}_k)_y - (\hat{z}_k)_{y_i})]_+.
 \end{aligned} \tag{11}$$

The inner maximization is precisely the optimization problem we considered earlier of finding the most extreme point within the adversarial polytope. If this objective can be minimized with zero training loss, then we are guaranteed that for every data point in the training set, no adversarial example is possible (all points within the adversarial polytope are labeled correctly by the classifier). Test points naturally have no such guarantee, but we can still use the method described above to test whether or not they are potentially adversarial. And if empirical generalization patterns for deep networks remain in this setting, it is likely that classifiers which don’t admit to adversarial examples on the training set may well have similar properties on the test set (and we will demonstrate this empirically ourselves shortly).

3 Convex outer bounds on the adversarial polytope

Despite all the promise of the preceding section, there is a notable disadvantage that renders this approach more a conceptual strategy rather than an actual tool: the fact that actually *solving* a mixed binary integer program is an NP-hard task. Although it is well-appreciated that many instances of reasonably-sized binary integer programs *are* empirically tractable via branch and cut algorithms, in this case the optimization problem of interest has a number of binary variables equal to the number of hidden units in the network, a number that can easily grow into the millions for large deep networks. This is simply not tractable to solve via exact solutions to the integer program.

Instead, the key claim of this section is that we can instead consider a convex relaxation of the adversarial polytope that is a strict *outer bound*. This concept is illustrated in Figure 2. That is, it contains all points (and more) within the adversarial polytope, but is a convex set and so can be optimized over efficiently (at least according to some definition of “efficient”, which will be a major discussion point throughout this section). The key point is that because of this outer bound property, *we can perform the same tests for potential adversarial examples or develop the same robust optimization techniques, which will provide identical guarantees*. That is, if we look for the “most adversarial” example over our outer approximation, and find that it does not change

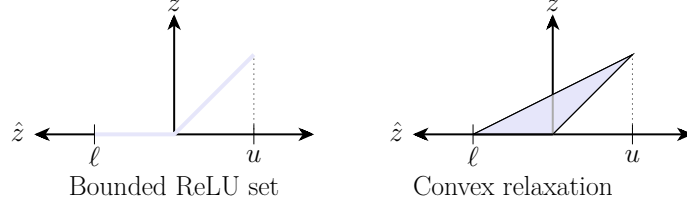


Figure 3: Illustration of the convex ReLU relaxation over the bounded set $[\ell, u]$.

the predicted class of an example, then we are guaranteed that no actual adversarial example can change the class either. Or we can develop robust classifiers that optimize over the worst-case loss within the outer approximation, and these (assuming zero training loss) also will guarantee that the classifier does not allow for adversarial examples.³

In this section we will first define our convex outer approximation to the adversarial polytope, and discuss the optimization problem that arises when optimizing over it. However, although this is a convex optimization problem and hence “tractable”, it is far from practically efficient to solve this optimization problem exactly for large networks. Thus, in the next section we consider the *dual problem* of optimization over the convex outer bound. By standard results in convexity, any dual feasible solution represents a *lower bound* on minimization over the convex adversarial approximation, and as we show, the dual problem itself can be formulated as a deep network that is effectively a type of “adjoint system” of the original network, and which easily allows us to attain dual feasible points; thus, we can employ simple gradient descent methods to optimize this dual problem, giving us guaranteed lower bounds on the original optimization problem, even if we don’t solve the dual problem to optimality. Next, we discuss how this dual framework can be fit into the context of learning a robust classifier, showing that we can indeed solve the robust classifier learning problem using the dual problem. Finally, we discuss methods for finding the necessary upper and lower bounds needed for the convex outer approximation.

3.1 A convex outer bound

Note that the definition of our adversarial polytope involved two types of terms: the linear terms $\hat{z}_{i+1} = W_i z_i + b_i$ (which are convex), and the non-convex ReLU terms $z_i = \max\{0, \hat{z}_i\}$. Thus, to form a convex relaxation of this set, we need to replace the ReLU terms by some relaxed version.

The idea that underlies our convex outer bound is quite simple, and indeed has been previously considered in the context of searching over the actual adversarial polytope Ehlers [2017]; however, as mentioned above, this previous work focused on the outer bound as a useful step in searching over the true adversarial polytope for a given classifier, while we consider it here as an outer bound that we will explicitly train over. However, we do emphasize that the formalism in this section at the single activation level is virtually identical to that previously proposed in this prior work, though because our full bound will also involve iterative computation of upper and lower bounds for intermediate activations, our final outer bound ends up being quite different.

Suppose that for some pre-ReLU activation $\hat{z}$ and adversarial polytope $\mathcal{Z}_\epsilon(x)$ we have an upper and lower bound ℓ and u over the allowable values for the activation (in the remainder of this

³Note, however, that the outer approximation is less useful for actually find adversarial examples: as we will see, because elements in this polytope may not correspond to any actual path through the original network, it is unclear what if anything can be said about the actual “examples” found via this method. However, given the relative ease of finding versus preventing adversarial examples in modern deep architectures, this is not overly bothersome

paragraph, for simplicity, we'll let $\hat{z}$ and z denote scalar values for a *single* activation, which we'll later extend elementwise to all activations). Note that these upper and lower bounds are guaranteed to exist, as perturbation of ϵ of the input can only change the activations of the network by some bounded amount (this is effectively the “adversarial” polytope but for inner activations of the network). Then we can replace the non-convex ReLU constraint by its convex hull over the allowable upper and lower bounds. This reduces to three cases:

1. If $\ell \leq u \leq 0$, then we can replace the constraint $z = \max\{0, \hat{z}\}$ with the constraint $z = 0$, as the ReLU will always lie in the zero region.
2. Similarly, if $0 \leq \ell \leq u$, then we can replace the constraint with $z = \hat{z}$, as the ReLU will always lie in the linear region.
3. In the remaining case (the “interesting” non-convex case), where $\ell < 0 < u$, we can replace the non-convex ReLU constraint with the convex hull over the ReLU function over the domain $[\ell, u]$. This is shown in Figure 3, and the constraint set in this case can be represented by the three linear inequalities

$$z \geq 0, \quad z \geq \hat{z}, \quad -u\hat{z} + (u - \ell)z \leq -u\ell. \quad (12)$$

With this relaxation as our starting point, we can formally define our relaxation of the adversarial polytope, which we denote $\tilde{\mathcal{Z}}_e^{[\ell, u]}(x)$ (here ℓ and u signify tensors of upper and lower bounds for each activation in the network, which are naturally going to be different for different activations in the network, and which we discuss how to compute shortly) as the following set of linear constraints

$$\begin{aligned} & \hat{z}_{i+1} = W_i z_i + b_i, \quad i = 1, \dots, k-1 \\ & z_1 \leq x + \epsilon \\ & z_1 \geq x - \epsilon \\ & z_{i,j} = 0, \quad i = 2, \dots, k-1, j \in \mathcal{I}_i^- \\ & z_{i,j} = \hat{z}_{i,j}, \quad i = 2, \dots, k-1, j \in \mathcal{I}_i^+ \\ & \left. \begin{aligned} & z_{i,j} \geq 0, \\ & z_{i,j} \geq \hat{z}_{i,j}, \\ & -u_{i,j}\hat{z}_{i,j} + (u_{i,j} - \ell_{i,j})z_{i,j} \leq -u_{i,j}\ell_{i,j} \end{aligned} \right\} \quad i = 2, \dots, k-1, j \in \mathcal{I}_i \end{aligned} \quad (13)$$

where $\mathcal{I}_i^-$, $\mathcal{I}_i^+$, and $\mathcal{I}_i$ denote the set of activations in the i th layer that fit the categories 1, 2, and 3 above respectively. Intuitively, our convex outer approximation allows some flexibility in choosing activations as subsequent layers of the network. For activations that can span over zero, the “post-ReLU” activation need not exactly equal the ReLU of the input, but can lie anywhere between the ReLU and the line connecting the maximum to minimum values; this extra freedom allows us to pick examples that are “more adversarial” at later layers of the network, because they may not correspond to any actual input fed through the network, but instead will correspond to allowing the intermediate activations in a network to be adjusted to make the example as adversarial as possible.

At this point, we could theoretically use our convex outer approximation in lieu of the true adversarial polytope, and obtain a tractable formulation to identify potential adversarial examples or learn a robust classifier. However, two key challenges arise in this approach:

1. We need a way of optimizing over the adversarial polytope efficiently, that is, solving the optimization problem

$$\text{minimize } c^T \hat{z}_k, \quad \text{subject to } \hat{z}_k \in \tilde{\mathcal{Z}}_e^{[\ell, u]}(x). \quad (14)$$

Although this is now a convex linear program, it still has the number of variables equal to the number of hidden units in the network, and standard solution methods for LPs would be intractable for all but very small networks.

2. We need some way of finding the lower and upper bounds ℓ and u . Simple Lipschitz bounds are extremely loose here (a fact we will demonstrate shortly), and provide no real use. We need some alternative way of finding these bounds that will eventually provide a meaningful convex outer bound.

3.2 Efficient optimization via the dual problem

In this section we deal with the first of the two problems above, and provide an efficient method for solving (or more precisely, for bounding the optimal value of) the optimization problem above, written explicitly as

$$\begin{aligned}
& \text{minimize} && c^T \hat{z}_k \\
& \text{subject to} && \hat{z}_{i+1} = W_i z_i + b_i, \quad i = 1, \dots, k-1 \\
& && z_1 \leq x + \epsilon \\
& && -z_1 \leq -x + \epsilon \\
& && z_{i,j} = 0, \quad i = 2, \dots, k-1, j \in \mathcal{I}_i^- \\
& && z_{i,j} = \hat{z}_{i,j}, \quad i = 2, \dots, k-1, j \in \mathcal{I}_i^+ \\
& && \left. \begin{aligned} & -z_{i,j} \leq 0, \\ & \hat{z}_{i,j} - z_{i,j} \leq 0, \\ & -u_{i,j} \hat{z}_{i,j} + (u_{i,j} - \ell_{i,j}) z_{i,j} \leq -u_{i,j} \ell_{i,j} \end{aligned} \right\} \quad i = 2, \dots, k-1, j \in \mathcal{I}_i
\end{aligned} \tag{15}$$

The basic idea we propose is to consider the dual of this linear program. By standard results in convex analysis, any feasible solution of the dual problem provides a guaranteed lower bound on the optimal objective of the primal problem. Crucially, we will show that the dual problem can be characterized via a “dual network”, similar to the backpropagation network through the original graph, but with additional free parameters that can be optimized over with e.g. gradient descent. And although we don’t expect to find an optimal solution to the dual problem in this manner, it will still provide the guarantee we need.

In detail, we associate the following dual variables with each of the constraints

$$\begin{aligned}
\hat{z}_{i+1} = W_i z_i + b_i &\Rightarrow \nu_{i+1} \in \mathbb{R}^{|\hat{z}_{i+1}|} \\
z_1 \leq x + \epsilon &\Rightarrow \xi^+ \in \mathbb{R}^{|x|} \\
-z_1 \leq -x + \epsilon &\Rightarrow \xi^- \in \mathbb{R}^{|x|} \\
-z_{i,j} \leq 0 &\Rightarrow \mu_{i,j} \in \mathbb{R} \\
\hat{z}_{i,j} - z_{i,j} \leq 0 &\Rightarrow \tau_{i,j} \in \mathbb{R} \\
-u_{i,j} \hat{z}_{i,j} + (u_{i,j} - \ell_{i,j}) z_{i,j} \leq -u_{i,j} \ell_{i,j} &\Rightarrow \lambda_{i,j} \in \mathbb{R}
\end{aligned} \tag{16}$$

where we note that can easily eliminate the dual variables corresponding to the $z_{i,j} = 0$ and $z_{i,j} = \hat{z}_{i,j}$ from the optimization problem, so we don’t define explicit dual variables for these; we also note that $\mu_{i,j}$, $\tau_{i,j}$, and $\lambda_{i,j}$ are only defined for i, j such that $j \in \mathcal{I}_i$, but we keep the notation as above for simplicity. With these definitions, the dual problem becomes

$$\begin{aligned}
& \text{maximize} && - \sum_{i=1}^{k-1} \nu_{i+1}^T b_i - (x + \epsilon)^T \xi^+ + (x - \epsilon)^T \xi^- + \sum_{i=2}^{k-1} \lambda_i^T (u_i \ell_i) \\
& \text{subject to} && \nu_k = -c \\
& && \nu_{i,j} = 0, \quad j \in \mathcal{I}_i^- \\
& && \nu_{i,j} = (W_i^T \nu_{i+1})_j, \quad j \in \mathcal{I}_i^+ \\
& && \left. \begin{aligned} (u_{i,j} - \ell_{i,j}) \lambda_{i,j} - \mu_{i,j} - \tau_{i,j} &= (W_i^T \nu_{i+1})_j \\ \nu_{i,j} &= u_{i,j} \lambda_{i,j} - \mu_i \end{aligned} \right\} i = 2, \dots, k-1, j \in \mathcal{I}_i, \\
& && W_1^T \nu_2 = \xi^+ - \xi^- \\
& && \lambda, \tau, \mu, \xi^+, \xi^- \geq 0
\end{aligned} \tag{17}$$

The key insight we highlight here is that *the dual problem can also be written in the form of a deep network*, which provides a trivial way to find feasible solutions to the dual problem, which can then be optimized over. Specifically, consider the constraints

$$\begin{aligned}
(u_{i,j} - \ell_{i,j}) \lambda_{i,j} - \mu_{i,j} - \tau_{i,j} &= (W_i^T \nu_{i+1})_j \\
\nu_{i,j} &= u_{i,j} \lambda_{i,j} - \mu_i.
\end{aligned} \tag{18}$$

Note that the dual variable λ corresponds to the upper bounds in the convex ReLU relaxation, while μ and τ correspond to the lower bounds $z \geq 0$ and $z \geq \hat{z}$ respectively; by the complementarity property, we know that at the optimal solution, these variables will be zero if the ReLU constraint is non-tight, or non-zero if the ReLU constraint is tight. Because we cannot have the upper and lower bounds be simultaneously tight (this would imply that the ReLU input $\hat{z}$ would exceed its upper or lower bound otherwise), we know that either λ or $\mu + \tau$ must be zero. This means that at the optimal solution to the dual problem

$$\begin{aligned}
(u_{i,j} - \ell_{i,j}) \lambda_{i,j} &= [(W_i^T \nu_{i+1})_j]_+ \\
\tau_{i,j} + \mu_{i,j} &= [(W_i^T \nu_{i+1})_j]_-
\end{aligned} \tag{19}$$

i.e., the dual variables capture the positive and negative portions of $(W_i^T \nu_{i+1})_j$ respectively. Combining this with the constraint that

$$\nu_{i,j} = u_{i,j} \lambda_{i,j} - \mu_i \tag{20}$$

means that

$$\nu_{i,j} = \frac{u_{i,j}}{u_{i,j} - \ell_{i,j}} [(W_i^T \nu_{i+1})_j]_+ - \alpha [(W_i^T \nu_{i+1})_j]_-, \quad \text{for } j \in \mathcal{I}_i \tag{21}$$

for some $0 \leq \alpha \leq 1$ (this accounts for the fact that we can either put the “weight” of $[(W_i^T \nu_{i+1})_j]_-$ into μ or τ , which will or will not be passed to the next ν_i). This is exactly a type of leaky ReLU operation, with a slope in the positive portion of $u_{i,j}/(u_{i,j} - \ell_{i,j})$ (a term between 0 and 1), and a negative slope anywhere between 0 and 1. Similarly, and more simply, note that ξ^+ and ξ^- simply denote the positive and negative portions of $W_1^T \nu_2$, so we can replace these terms with an absolute value in the objective. Finally, we note that although it is possible to have $\mu_{i,j} > 0$ and $\tau_{i,j} > 0$ simultaneously, this corresponds to an activation that is identically zero pre-ReLU (both constraints being tight), and so is expected to be relatively rare. Putting this all together, and

using $\hat{\nu}$ to denote “pre-activation” variables in the dual network, we can write the dual problem in terms of the network

$$\begin{aligned} \nu_k &= -c \\ \hat{\nu}_i &= W_i^T \nu_{i+1}, i = k-1, \dots, 1 \\ \nu_{i,j} &= \begin{cases} 0 & j \in \mathcal{I}_i^- \\ \hat{\nu}_{i,j} & j \in \mathcal{I}_i^+ \\ \frac{u_{i,j}}{u_{i,j} - \ell_{i,j}} [\hat{\nu}_{i,j}]_+ - \alpha_{i,j} [\hat{\nu}_{i,j}]_- & j \in \mathcal{I}_i \end{cases} \end{aligned} \quad (22)$$

which we will abbreviate as $\nu = g_{\theta,x}(c, \alpha)$ to emphasize the fact that $-c$ acts as the “input” to the network and α are per-layer inputs we can also specify (for only those activations in $\mathcal{I}_i$), where ν in this case is shorthand for all the ν_i and $\hat{\nu}_i$ activations. It is worth noting that this is exactly the backpropagation for our original classifier (the ν variables are the backprop gradients, and the $\nu_{i,j}$ terms are set to be either zero or $\hat{\nu}_{i,j}$ if we are on the zero or linear portions of the ReLU respectively) except that the pass for the $\mathcal{I}_i$ variables has some additional free parameters: positive terms $\hat{\nu}_{i,j}$ are weighted by $\frac{u_{i,j}}{u_{i,j} - \ell_{i,j}}$ and for negative terms we have the freedom to choose a weighting between zero and one via the free parameter α .

The final objective we are seeking to optimize can also be written

$$\begin{aligned} J(\nu) &= - \sum_{i=1}^{k-1} \nu_{i+1}^T b_i - (x + \epsilon)^T [\hat{\nu}_1]_+ + (x - \epsilon)^T [\hat{\nu}_1]_- + \sum_{i=2}^{k-1} \sum_{j \in \mathcal{I}_i} \frac{u_{i,j} \ell_{i,j}}{u_{i,j} - \ell_{i,j}} [\hat{\nu}_{i,j}]_+ \\ &= - \sum_{i=1}^{k-1} \nu_{i+1}^T b_i - x^T \hat{\nu}_1 - \epsilon \|\hat{\nu}_1\|_1 + \sum_{i=2}^{k-1} \sum_{j \in \mathcal{I}_i} \ell_{i,j} [\nu_{i,j}]_+ \end{aligned} \quad (23)$$

Thus our final rewritten form of the dual problem is

$$\text{maximize } J(g(c, \alpha)) \quad (24)$$

where we optimize over the variables α , and where we can solve the problem using any standard deep learning optimization method such as gradient descent (projected gradient descent, as there are the constraints that $0 \leq \alpha_{i,j} \leq 1$). Furthermore, the above consideration that it is unlikely for $\tau_{i,j}$ and $\mu_{i,j}$ to simultaneously be 0, means that we expect α to attain values of either 0 or 1. This means we can even apply methods like the fast gradient sign to simply observe the gradient w.r.t. $\alpha_{i,j}$ at some intermediate value (e.g. 0.5), and then move it to either 0 or 1 depending on the sign of its gradient, in order to provide a quick bound with just a single optimization iteration.

A crucial point here is that this formulation of the problem is *not* convex in α , and so in general we have no guarantee of attaining optimal solutions. However, the key point is that *because the dual network always corresponds to feasible solutions of the dual problem, any solution will be a guaranteed lower bound on the optimal objective of the primal.* Thus, as long as the optimization procedure works well in practice (which we know to be typically the case for deep network optimization), we generally expect good performance from the method, i.e., that it will produce good bounds on the optimal solution of the primal.

3.3 Determining activation bounds

We have thus far ignored the question of how to actually find bounds ℓ_i and u_i on the pre-ReLU activations in the network. The discussion above, however, motivates a simple approach to doing so. We can simply build these bounds iteratively, layer by layer, using precisely the optimization

problem formulated above. For instance, suppose that we have determined bounds for layers $2, \dots, n$; then we can solve the above optimization problem with $c = e_i$ or $c = -e_i$ to find lower and upper bounds respectively for the i th activation in the $n + 1$ layer.⁴ Note again that because we solve this optimization problem via the dual, these bounds are guaranteed to be strict, even if we don't solve the dual problem to optimality.

Naturally, solving two optimization problems per activation in the network (per example) would be a daunting task, and in this section we present alternative approaches that still give guaranteed bounds but which are much more efficient than this "brute force" approach. However, we fully admit that currently this step is by far the least scalable element of our overall approach, and finding the proper approximation schemes to scale to e.g. ImageNet-sized systems remains an open challenge.

The basis of our approximation approach here is precisely the fact we mentioned before: that even suboptimal solutions of the dual optimization problem are guaranteed to give valid bounds. Thus, instead of optimizing over α for each activation (and each example), we use a specific fixed feasible solution for α that substantially simplifies the problem. In particular, choosing

$$\alpha_{i,j} = \frac{u_{i,j}}{u_{i,j} - \ell_{i,j}} \quad (25)$$

guarantees that the "slope" of the leaky ReLU for those elements $j \in \mathcal{I}_i$ is the same in the negative portion as the positive portion, i.e., the operation is simply linear (note that $0 < \alpha_{i,j} < 1$ since $\ell_{i,j}$ is negative). Then the value of ν_i for all activations simultaneously (though still for a single example, since ℓ and u are sample-dependent), is given by

$$\begin{aligned} \hat{\nu}_i &= W_i^T D_{i+1} W_{i+1}^T \dots D_n W_n^T \\ \nu_i &= D_i \hat{\nu}_i \end{aligned} \quad (26)$$

where again, we suppose that we have thus far already computed bounds up until layer n , and where D_i is a diagonal matrix with

$$(D_i)_{jj} = \begin{cases} 0 & j \in \mathcal{I}_i^- \\ 1 & j \in \mathcal{I}_i^+ \\ \frac{u_{i,j}}{u_{i,j} - \ell_{i,j}} & j \in \mathcal{I}_i \end{cases} . \quad (27)$$

Now we consider computing the objective value $J(\nu)$ for this particular setting of ν . The two linear terms in the objective

$$x^T \hat{\nu}_1 \quad \text{and} \quad \sum_{i=1}^{k-1} b_i^T \nu_{i+1} \quad (28)$$

are both simple to handle in the above case: we simply perform the multiplication with ν_i left-to-right, which corresponds simply to feeding the initial example and the bias terms through the network (without bias terms).

The $\epsilon \|\nu\|_1$ term in the objective is harder to deal with; because this is a non-linear term, in general there is nothing to do except form the entire ν_1 matrix explicitly, then compute the ℓ_1 norm of each column. This still may not be entirely intractable: we can perform the matrix multiplication (26) in any order we want, so if the input to the network is substantially smaller than the number

⁴Although we have discussed using the procedure to generate lower bounds, it can also trivially produce upper bound on $\max c^T \hat{z}_k$ by minimizing $\min -c^T \hat{z}_k$; if J is a lower bound on the minimization problem, then $-J$ will be an upper bound on the maximization problem.

of activations at a particular layer, or if there is some “bottleneck” layer of limited dimension, such as a PCA pre-processing of the data, then computing the full ν_1 matrix may be reasonable. Alternatively, when the linear operators are highly structured (such as convolutions), there will be a large amount of sparsity that can be exploited, at least for networks that are not too deep; we could also use random projection methods [Candes and Tao, 2006] to derive high-probability bounds on the norms in question; or we can even use an optimization formulation to in turn bound *this* quantity via convex duality. However, we leave a thorough investigation of these approaches for future work, and focus here are cases where we simply compute the matrix exactly.

The same considerations hold true for the $\ell_{i,j}[\nu_{i,j}]_+$ terms in the objective: they are non-linear so cannot easily be simplified. However, in this case we only need to compute these terms for i, j such that $j \in \mathcal{I}_j$, which is hopefully a manageably small subset of the total number of activations (if it is not, the convex outer adversarial polytope is likely to be extremely large anyway).

With these considerations in mind, we propose a layer-by-layer method for generating all the ℓ_i and u_i bounds: we start by generating lower and upper bounds for z_2 by the above procedure (which actually reduces to just a simple norm bound in this case), use these bounds to generate upper and lower bounds for z_3 , etc. Several terms can be reused through this computation, and we highlight a reasonably efficient method for doing so (though still explicitly maintaining $\hat{\nu}_1$ and the various $\nu_{i,j}$ terms for $j \in \mathcal{I}_i$) in Algorithm 1.

It is worth highlighting the connection between this incremental approach and standard norm bounds. Note that for the first layer in the network, all terms effectively lie in the “linear” set $\mathcal{I}_1^+$. Thus we have $\hat{\nu}_1 = -W_1^T$, and are upper and lower bounds $J(\nu)$ reduce to

$$\begin{aligned}\ell_2 &= x^T W_1^T + b_1^T - \epsilon \|W_1^T\|_1 \\ u_2 &= x^T W_1^T + b_1^T + \epsilon \|W_1^T\|_1\end{aligned}\tag{29}$$

where the first term is just the example x fed through the first layer of the network and the $\epsilon \|W_1^T\|_1$ (where for a matrix this denotes the column-wise ℓ_1 norm) is a standard norm bound on how far a bounded ℓ_∞ perturbation can reach. For later layers, the first two terms are replaced by

$$x^T \hat{\nu}_1 + \sum_{i=1}^{k-1} b_i^T \nu_{i+1}\tag{30}$$

which again just denotes the example and bias terms fed through the network, whereas the later term is replaced by

$$\epsilon \|\hat{\nu}_1\|_1 = \epsilon \|W_1^T D_2 W_2^T \dots D_n W_n^T\|_1\tag{31}$$

which is just the ℓ_1 norms of the actual product of weights and activation terms. This bound would be sufficient if all activations lay in $\mathcal{I}^-$ and $\mathcal{I}^+$, i.e. if the network could only be perturbed within a linear region. But obviously in reality the perturbation can cause some activations to switch between positive and negative (exactly the $\mathcal{I}_i$ set) and in these cases we pay an additional “cost” in our bounds of $\ell_{i,j}[\nu_{i,j}]_+$. Remember that $\nu_{i,j}$ positive at the optimal solution corresponds to activations that “cheat” by lying on the upper portion of the convex ReLU relaxation, and the bound nicely capture the fact that it will be looser in this case.

As a final note, we highlight the fact that the incremental computation of all these bounds, followed by some number of optimization steps for the final dual problem, can all be implemented within an automatic differentiation library (this entire procedure really is what defines our actual outer bound on the adversarial polytype). This means that the final lower bound on $c^T \hat{z}_k$ can be differentiated with respect to the parameters of the network itself. Thus, in learning our classifier we will explicitly minimize over the entire computation of this bound.

Algorithm 1 Compute network bounds

input: Network parameters $\{W_i, b_i\}_{i=1}^{k-1}$, data point x , ball size ϵ
// initialization
 $\hat{\nu}_1 := -W_1^T$
 $\gamma_1 := -b_1^T$
 $\ell_2 := x^T W_1^T + b_1^T - \epsilon \|W_1^T\|_1$ // $\|\cdot\|_1$ for a matrix here denotes ℓ_1 norm of all columns
 $u_2 := x^T W_1^T + b_1^T + \epsilon \|W_1^T\|_1$
for $i = 2, \dots, k-1$ **do**
 form $\mathcal{I}_i^-, \mathcal{I}_i^+, \mathcal{I}_i$; form D_i as in (27)
 // initialize new terms
 $\nu_{i, \mathcal{I}_i} := (D_i)_{\mathcal{I}_i} W_i^T$
 $\gamma_i := -b_i^T$
 // propagate existing terms
 $\nu_{j, \mathcal{I}_j} := \nu_{j, \mathcal{I}_j} D_i W_i^T, \quad j = 2, \dots, i-1$
 $\gamma_j := \gamma_j D_i W_i^T, \quad j = 1, \dots, i-1$
 $\hat{\nu}_1 := \hat{\nu}_1 D_i W_i^T$
 // compute bounds
 $\ell_i := x^T \hat{\nu}_1 + \sum_{j=1}^i \gamma_j - \epsilon \|\hat{\nu}_1\|_1 + \sum_{j=2}^i \ell_i[\nu_{i, \mathcal{I}_i}]_+$
 $u_i := x^T \hat{\nu}_1 + \sum_{j=1}^i \gamma_j + \epsilon \|\hat{\nu}_1\|_1 - \sum_{j=2}^i \ell_i[-\nu_{i, \mathcal{I}_i}]_+$
end for
output: bounds $\{\ell_i, u_i\}_{i=2}^{k-1}$

3.4 Integration with robust optimization

Finally, we highlight how the above approach can be easily integrated within the robust classifier learning objective we highlighted before. Recall that our robust optimization problem was given by

$$\underset{\theta}{\text{minimize}} \quad \sum_{i=1}^N [1 + \max_{y \neq y_i} \max_{\hat{z} \in \tilde{\mathcal{Z}}_\epsilon} (x_i)((\hat{z}_k)_y - (\hat{z}_k)_{y_i})]_+. \quad (32)$$

Using the techniques from this section, we can bound this term as

$$\begin{aligned} \min_{\theta} \sum_{i=1}^N [1 + \max_{y \neq y_i} \max_{\hat{z} \in \tilde{\mathcal{Z}}_\epsilon} (x_i)((\hat{z}_k)_y - (\hat{z}_k)_{y_i})]_+ &\leq \min_{\theta} \sum_{i=1}^N [1 + \max_{y \neq y_i} \max_{\hat{z} \in \tilde{\mathcal{Z}}_\epsilon(x_i)} ((\hat{z}_k)_y - (\hat{z}_k)_{y_i})]_+ \\ &\leq \min_{\theta, \alpha_{1:N}} \sum_{i=1}^N [1 - \max_{y \neq y_i} J(g_{\theta, x_i}(e_y - e_{y_i}, \alpha_i))]_+ \end{aligned} \quad (33)$$

i.e., we can substitute the cost function from our dual network into the loss of our original optimization problem, then solve simultaneously over θ and α_i . Note that in this formulation, computing our dual network $\nu = g_{\theta, x_i, \epsilon}(c, \alpha_i)$ implicitly also means computing the sequential bounds as described in the previous section, and we use the subscripts x_i, ϵ on g to emphasize the fact that these bounds (and hence the dual network) depends on the actual example and ϵ . In practice, during training we often forgo optimization over α and simply use the solution mentioned in the previous section to compute the upper and lower bounds when computing the final bound as well.

Lastly, although the hinge loss is useful for illustrating the connection between the loss function and the optimization problem, in practice it is less effective than more common losses such as cross

entropy. Fortunately, for any convex monotonic loss function (which includes cross entropy), we can use the bound

$$\max_{\|\Delta\|_1 \leq \epsilon} \ell(f_\theta(x_i + \Delta), y_i) \leq \ell(-J(g_{\theta, x_i}(I - e_{y_i} 1^T, \alpha_i)), y_i) \quad (34)$$

to use more general loss functions, where $-J(g_{\theta, x}(I - e_y 1^T, \alpha_i))$ (i.e., the dual bound evaluated at $c = e_y - e_{y_i}$ for all y) takes the place of the activations.

3.5 Summary and discussion

The presentation in this section was quite lengthy, so it is useful to provide a summary of the end result. The final outcome of our approach is that if we learn our network by solving the optimization problem

$$\underset{\theta, \alpha}{\text{minimize}} \quad \sum_{i=1}^N \ell(-J(g_{\theta, x_i, \epsilon}(I - e_{y_i} 1^T, \alpha_i)), y_i) \quad (35)$$

where g_{θ, x_i} is a network we compute using a few passes through the original network and its backpropagation network (albeit with much larger “batch” sizes through the network, since in the worst case we need to feed in a sample for each dimension of the input to compute the ℓ and u bounds), then this provides a *guaranteed* bound on the adversarial loss we can suffer for *any* adversarial perturbation with ℓ_∞ norm bounded by ϵ . The method also produces a guaranteed bound on the adversarial error, by simply checking whether the y_i coordinate of $-J(g_{\theta, x_i, \epsilon}(I - e_{y_i} 1^T, \alpha_i))$ is the largest (by the scaling with the input $c = I - e_{y_i} 1^T$, this coordinate is always zero, so it amounts to checking whether all other coordinates are negative).

Although the method is based upon convex duality and linear programming, no recourse to an actual linear programming solver is needed, nor is any iterative method for solving any inner optimization problems; we simply optimize the entire (nonconvex) objective with stochastic gradient descent or any other of its variants used in deep learning, and we get the guaranteed bound.

Finally, we note that all the bounds and guarantees we have discussed so far apply to the training data (they can be evaluated on a test set, but like all test error, only if the test labels are known). What are we to do with such networks in deployment, where we see new examples without knowledge of the actual label? Fortunately, by the transitivity of adversarial examples, we can use the same technique to determine whether the example *might* be adversarial. Specifically, given a new example x we compute the network prediction $\hat{y} = f(x)$, then determine whether there is any potential adversarial example for this predicted label within ϵ . If not, then the example must *not* be adversarial, because there is no point within ϵ that changes the class prediction (i.e., there couldn’t be a “normal” input ϵ away from this possibly adversarial example). Obviously, this approach may sometimes classify non-adversarial inputs as potentially adversarial, but it has zero false negatives, in that it will never fail to flag an adversarial example. Given the challenge in even defining adversarial examples in general, this seems to be as strong a guarantee as is currently possible.

4 Experimental results

Here we present experiments on small-scale problems designed to demonstrate the approach. Although the method does not yet scale to ImageNet-sized classifiers, we do demonstrate the approach on a simple convolutional network applied to MNIST, illustrating that the method can apply to approaches beyond fully-connected networks with very small sizes (which represent the state of the

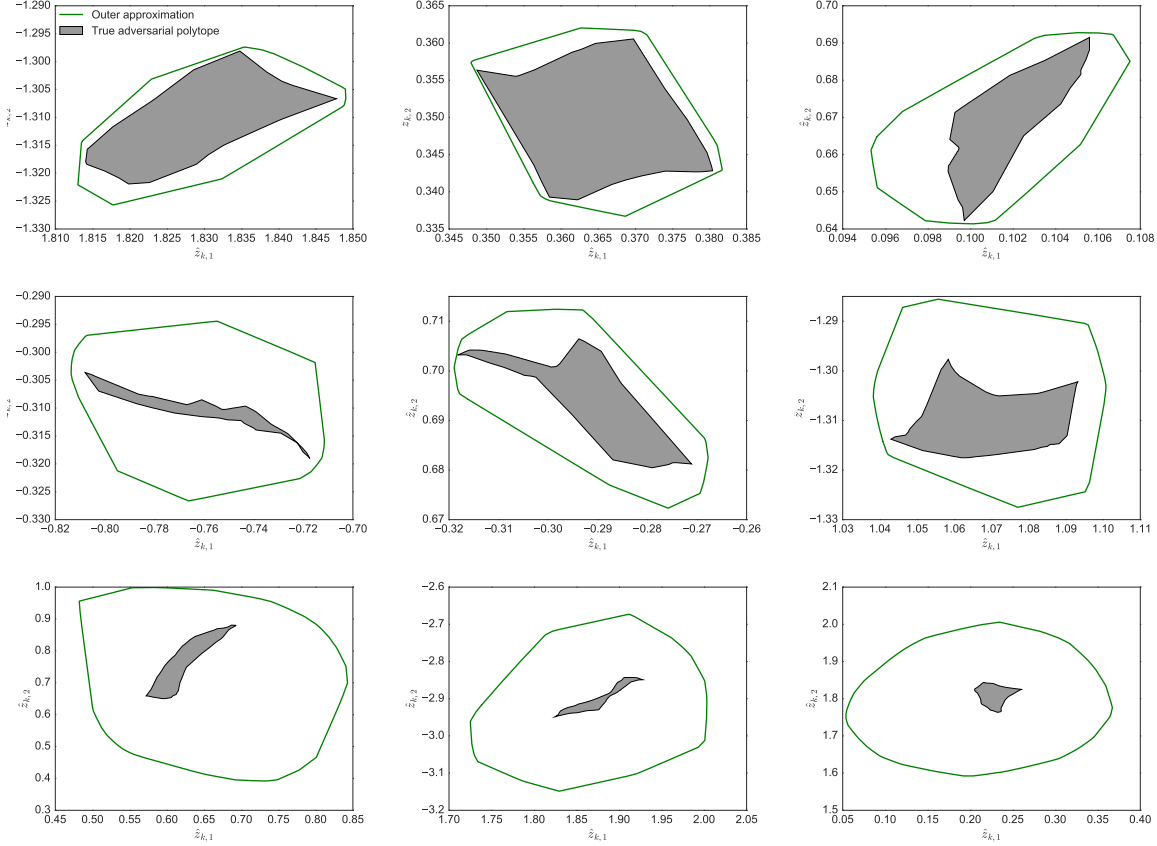


Figure 4: Illustrations of the true adversarial polytope (gray) and our convex outer approximation (green) for a random 2-100-100-100-100-2 network with $\mathcal{N}(0, 1/\sqrt{n})$ weight initialization. Polytopes are shown for $\epsilon = 0.05$ (top row), $\epsilon = 0.1$ (middle row), and $\epsilon = 0.25$ (bottom row).

art for most existing work on neural network verification). Scaling challenges were discussed briefly above, and we highlight them more below. As mentioned above, code for the examples included in the paper is available at http://github.com/locuslab/convex_adversarial.

4.1 2D toy domain

Our first set of experiments involves a simple domain where both the input and output spaces of the network are two dimensional, so can be easily visualized. Specifically, we consider a 2-100-100-100-100-2 fully connected network.

Visualizations of the convex outer adversarial polytope To begin, we consider some simple cases of visualizing the outer approximation to the adversarial polytope for random networks. Because the output space is two-dimensional we can easily visualize the polytopes in the output layer, and because the input space is two dimensional, we can easily cover the entire input space densely to enumerate the true adversarial polytope. In this experiment, we initialized the weights of the all layers to be normal $\mathcal{N}(0, 1/\sqrt{n_{\text{in}}})$ and biases normal $\mathcal{N}(0, 1)$ (due to scaling, the actual absolute value of weights is not particularly important except as it relates to ϵ). Although obviously

not too much should be read into these experiments with random networks, the main takeaways are that 1) for “small” ϵ , the outer bound is an extremely good approximation to the adversarial polytope; 2) as ϵ increases, the bound gets substantially weaker. This is to be expected: for small ϵ , the number of elements in $\mathcal{I}$ will also be relatively small, and thus additional terms that make the bound lose are expected to be relatively small (in the extreme, when no activation can change, the bound will be exact, and the adversarial polytope will be a convex set). However, as ϵ gets larger, more activations enter the set $\mathcal{I}$, and the available freedom in the convex relaxation of each ReLU increases substantially, making the bound looser. Naturally, the question of interest is how tight this bound is for networks that are actually trained to minimize the robust loss, which we will look at shortly.

Comparison to naive layerwise bounds One additional point is worth making in regards to the bounds we propose. It would also be possible to achieve a naive “layerwise” bound by iteratively determining absolute allowable ranges for each activation in a network (via the simple norm bound mentioned above), then for future layers, assuming each activation can vary arbitrarily within this range. This provides a simple iterative formula for computing layer-by-layer absolute bounds on the coefficients, and similar techniques have been used e.g. in Parseval Networks [Cisse et al., 2017] to produce more robust classifiers (albeit there considering ℓ_2 perturbations instead of ℓ_∞ perturbations, which likely are better suited for such an approach). Unfortunately, these naive bounds are extremely loose for multi-layer networks (in the first hidden layer, they naturally match our bounds exactly). For instance, for the adversarial polytope shown in Figure 4 (top left), the actual adversarial polytope is contained within the range

$$\hat{z}_{k,1} \in [1.81, 1.85], \quad \hat{z}_{k,2} \in [-1.33, -1.29] \quad (36)$$

with the convex outer approximation mirroring it rather closely. In contrast, the layerwise bounds produce the bound:

$$\hat{z}_{k,1} \in [-11.68, 13.47], \quad \hat{z}_{k,2} \in [-16.36, 11.48]. \quad (37)$$

Such bounds are essentially vacuous in our case, which makes sense intuitively. The naive bound has no way to exploit the “tightness” of activations that lie entirely in the positive space, and effectively replaces the convex ReLU approximation with a (larger) box covering the entire space. Thus, such bounds are not of particular use when considering robust classification.

Robust classifier training Finally, we consider training a simple robust classifier in our 2D example. Specifically, we incrementally randomly sample 50 points within the $[0, 1]$ xy -plane, at each point waiting until we find a sample that is at least 0.08 away from other points via ℓ_∞ distance, and assign each point a random label. We then attempt to learn a robust classifier that will correctly classify all points with an ℓ_∞ ball of $\epsilon = 0.03$. Note that there is no notion of generalization here, we are just evaluating the ability of the learning approach to fit a classification function robustly.

Figure 5 shows the resulting classifiers produced by standard training over just the data points themselves (left) and robust training via our method (right). As expected, the standard training approach results in points that are classified differently somewhere within their ℓ_∞ ball of radius $\epsilon = 0.03$ (this is exactly an adversarial example for the training set). In contrast, also as expected (because our procedure is able to attain zero robust error), the robust training method provides a classifier that is guaranteed to classify all points within the balls correctly. We use the Adam optimizer [Kingma and Ba, 2015] (over the entire batch of samples) with a learning rate of 0.001.

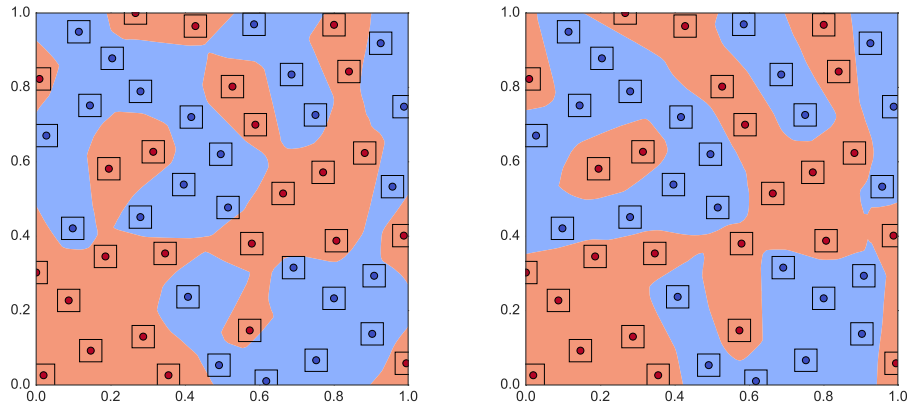


Figure 5: Illustration of classification boundaries resulting from standard training (left) and robust training (right) with ℓ_∞ balls of size $\epsilon = 0.03$ (shown in figure). The standard training procedure allows for some points within ball to have incorrect class labels, while the robust training does not (and the training procedure provides a bound verifying this fact).

Figure 6 (left) shows the training progress of the standard and robust learning processes on this example. Of note is the fact that the robust loss seems empirically “harder” to minimize, which is not particularly surprising since it effectively always looks for (a bound on) the worse case loss within the entire ℓ_∞ ball around the example. However, the network is able to eventually achieve zero robust error, which is expected given the fact that the data points can indeed be separated perfectly with a nonlinear classifier. Finally, it is of some interest to see what the true adversarial polytope for the examples in this data set looks like versus the convex approximation, evaluated at the solution of the robust optimization problem. Figure 6 (right) shows one of these figures, highlighting the fact that for the final network weights and choice of epsilon, the outer bound is empirically quite tight in this case.

4.2 MNIST ConvNet

Finally, we present results on producing a provably robust classifier on the MNIST data set. Specifically, we consider a ConvNet architecture that includes two convolutional layers, with 16 and 32 channels (each with a stride of two, to decrease the resolution by half without requiring max pooling layers), and two fully connected layers stepping down to 100 and then 10 (the output dimension) hidden units, with ReLUs following each layer except the last. That is, the network has the form:

$$x \Rightarrow \text{Conv_16x4x4,s=2} \Rightarrow \text{ReLU} \Rightarrow \text{Conv_16x4x4,s=2} \Rightarrow \text{ReLU} \Rightarrow \text{FC100} \Rightarrow \text{ReLU} \Rightarrow y. \quad (38)$$

This is a reasonable if fairly small network for MNIST: training the network using standard training achieves a test error of about 1.2%. Adding layers like batch normalization and dropout easily push the error below 1%, but since we don’t integrate these yet into our robust framework, we consider just the naive network. We use the Adam optimizer [Kingma and Ba, 2015] with a learning rate of 0.001 (the default option) with no additional hyperparameter selection.

Figure 7 shows the training progress using our procedure with a robust softmax loss function and $\epsilon = 0.1$. The “robust error” and “robust loss” here are our bounds on the robust error and

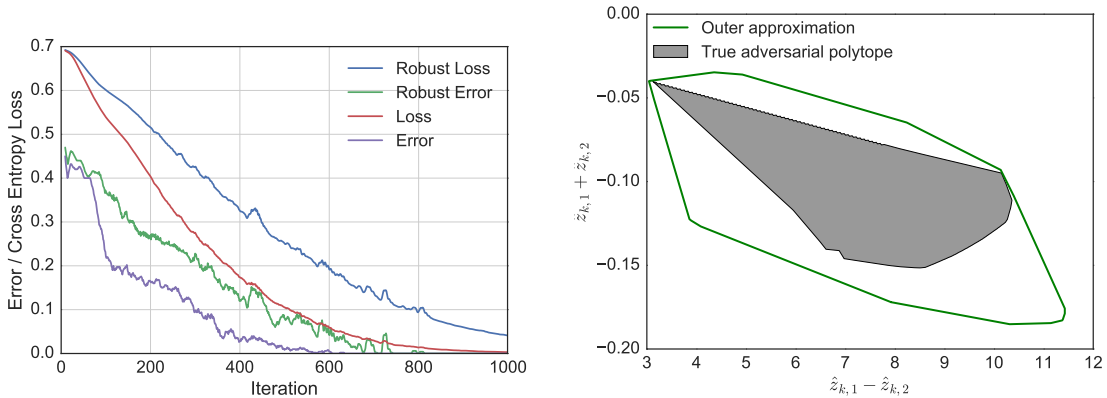


Figure 6: (left) Training curves for the 2D robust adversarial problem, where each iteration denotes a step of the Adam optimizer. (right) Illustration of the actual adversarial polytope and the convex outer approximation for one of the training points after the robust optimization procedure.

Network	Non-adversarial	FGSM	PGD	Robust Bound
Standard training	1.2%	39.7%	94.0%	100%
Robust training	2.9%	5.8%	6.2%	8.4%

Table 1: Adversarial errors for a traditionally-trained and our robustly-trained classifier on the MNIST test set.

loss; that is, we know that any norm-bounded adversarial technique will not be able to achieve loss or error that is any higher, though in practice it could be substantially lower as well. The final classifier after 20 epochs reaches a test error of 2.93% with a robust test error of 8.39%.

To see where other techniques would stand, we also tested our network against two common classes of attacks: the fast gradient sign method Goodfellow et al. [2015], and the projected gradient descent approach Madry et al. [2017].⁵ For a traditionally-trained classifier (with 1.2% test error) the FGSM approach results in 39.7% error, while PGD results in 94.0% error. On the classifier trained with our method, however, we only achieve 2.93% test error, but FGSM and PGD only achieve errors of 5.8% and 6.2% respectively (both, naturally, below our bound of 8.4%). These results are summarized in Table 1.

While this is by no means state of the art performance on standard MNIST (if such a thing really has any meaning at this point, but it is certainly valid to say that these results are decidedly *subpar* as far as generic MNIST classification goes), a few points stand out. First, this is by far the largest provably verified network we are currently aware of, and 8% error represents a reasonable performance given that it is against *any* adversarial attack strategy; the only other such bound we are aware of, from the very recent work mentioned previously ?, provides a guaranteed bound of 35% error. Second, in the example above training and testing error/loss are still tracking quite closely, suggesting that we should be able to further improve performance by simply using larger models. Therein lies the catch, though: the current model took 10 hours to train for 20 epochs on a Titan X (Maxwell) GPU. This is between two and three orders of magnitude more costly than

⁵For PGD, as in Madry et al. [2017] we use ℓ_∞ ball gradient descent and 100 iterations of step size 0.001, which was sufficient for convergence.

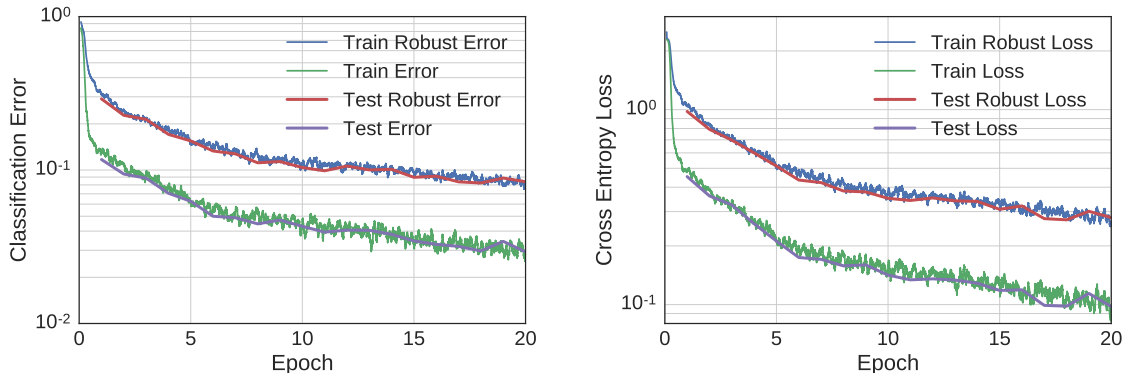


Figure 7: Training progress for our method applied to a simple ConvNet architecture on MNIST. Robust error and loss indicate provable upper bounds on the loss/error achievable by any adversarial perturbation with ℓ_∞ norm bounded by $\epsilon = 0.1$.

training the naive network (though the naive network, of course, is trivially susceptible to simple adversarial attacks like the PGD method). Building robust models within the framework to scale to e.g. ImageNet-sized image classification problems remains a challenging task. But because the approach is not combinatorial in its complexity, we believe it also represents a much more feasible approach than those based upon integer programming or satisfiability, which seem highly unlikely to ever scale to such problems. Thus, we believe the current performance represents a substantial step forward in research on adversarial examples in deep networks.

5 Conclusions

In this paper, we have presented a method for training classifiers that are provably robust to norm-bounded adversarial attacks. The method is based upon linear programming and duality theory, but crucially, does not actually involve solving a linear program or anything similarly costly. Instead, we design an objective that can be computed with the equivalent of a few passes through the original network and its backpropagation net (with larger batch sizes), yet still gives a guaranteed bound on the robust error and loss of the classifier.

While we feel this is a substantial step forward in the development of classifiers that are not susceptible to adversarial attacks, two main directions for improvement exist. First, as mentioned above, approaches are needed to make the method scale better. Naively, computing the upper and lower bounds as we described in this paper essentially require sending at least one sample through the network for every *dimension* of the input vector (and more at intermediate layers, if substantial numbers of activations have bounds that span zero). For domains like ImageNet, this is completely infeasible, and techniques such as (at a minimum) using bottleneck layers, other dual bounds, and probabilistic norm bounds are likely necessary to make the approach scale. However, unlike many past approaches, this scaling is not fundamentally combinatorial, so has some chance of success even in large networks.

Second, at some point it will be necessary to characterize attacks beyond simple norm bounds. While ℓ_∞ bounded examples are nice in that they offer a compelling visualization of images that look virtually “identical” to existing examples yet are misclassified, this is by no means the only set of possible attacks. For example, the recent work in Sharif et al. [2016] was able to break face

recognition software by using manufactured glasses, but adding glasses to a face image clearly is not bounded in ℓ_∞ norm. And even if we consider a norm where these examples *are* bounded (likely an ℓ_1 norm), then there almost certainly exist examples with the same ℓ_1 bound that would fool a human. Thus, the progress of adversarial examples must start to formalize what sorts of sets we will and won't allow for our adversarial perturbations, and these are almost certainly not going to be convex sets, as relied upon by our method. Thus, a great deal of work remains to understand both the space of adversarial examples that we *want* classifiers to be robust to, as well as methods for dealing with these likely highly non-convex sets in the *input* space.

Finally, although our focus in this paper was on adversarial examples and robust classification, the general techniques described here (namely optimizing over relaxed convex networks, and using a non-convex network representation of the dual problem to derive guaranteed bounds), may find applicability well beyond adversarial examples in deep learning. Many domain and problems such as inverting neural networks, neural style algorithms, and optimizing over latent spaces involve optimization problems that are a function of the neural network inputs or activations, and similar techniques may be brought to bear in these domains as well.

References

- Anonymous. Certified defenses against adversarial examples. In *Under review at ICLR 2018*, 2018.
- Anish Athalye and Ilya Sutskever. Synthesizing robust adversarial examples. *arXiv preprint arXiv:1707.07397*, 2017.
- Aharon Ben-Tal, Laurent El Ghaoui, and Arkadi Nemirovski. *Robust optimization*. Princeton University Press, 2009.
- Emmanuel J Candes and Terence Tao. Near-optimal signal recovery from random projections: Universal encoding strategies? *IEEE transactions on information theory*, 52(12):5406–5425, 2006.
- Nicholas Carlini and David Wagner. Towards evaluating the robustness of neural networks. In *Security and Privacy (SP), 2017 IEEE Symposium on*, pages 39–57. IEEE, 2017.
- Nicholas Carlini, Guy Katz, Clark Barrett, and David L Dill. Ground-truth adversarial examples. *arXiv preprint arXiv:1709.10207*, 2017.
- Moustapha Cisse, Piotr Bojanowski, Edouard Grave, Yann Dauphin, and Nicolas Usunier. Parseval networks: Improving robustness to adversarial examples. In *International Conference on Machine Learning*, pages 854–863, 2017.
- Ruediger Ehlers. Formal verification of piece-wise linear feed-forward neural networks. In *International Symposium on Automated Technology for Verification and Analysis*, 2017.
- Ian Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial examples. In *International Conference on Learning Representations*, 2015. URL <http://arxiv.org/abs/1412.6572>.
- Xiaowei Huang, Marta Kwiatkowska, Sen Wang, and Min Wu. Safety verification of deep neural networks. In *International Conference on Computer Aided Verification*, pages 3–29. Springer, 2017.

- Guy Katz, Clark Barrett, David Dill, Kyle Julian, and Mykel Kochenderfer. Reluplex: An efficient smt solver for verifying deep neural networks. *arXiv preprint arXiv:1702.01135*, 2017.
- Diederik Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015.
- Alessio Lomuscio and Lalit Maganti. An approach to reachability analysis for feed-forward relu neural networks. *arXiv preprint arXiv:1706.07351*, 2017.
- Jiajun Lu, Hussein Sibai, Evan Fabry, and David Forsyth. No need to worry about adversarial examples in object detection in autonomous vehicles. *arXiv preprint arXiv:1707.03501*, 2017.
- Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, and Adrian Vladu. Towards deep learning models resistant to adversarial attacks. *arXiv preprint arXiv:1706.06083*, 2017.
- Nicolas Papernot, Patrick McDaniel, Xi Wu, Somesh Jha, and Ananthram Swami. Distillation as a defense to adversarial perturbations against deep neural networks. In *Security and Privacy (SP), 2016 IEEE Symposium on*, pages 582–597. IEEE, 2016.
- Nicolas Papernot, Patrick McDaniel, Ian Goodfellow, Somesh Jha, Z Berkay Celik, and Ananthram Swami. Practical black-box attacks against deep learning systems using adversarial examples. In *Proceedings of the 2017 ACM Asia Conference on Computer and Communications Security*, 2017.
- Mahmood Sharif, Sruti Bhagavatula, Lujo Bauer, and Michael K Reiter. Accessorize to a crime: Real and stealthy attacks on state-of-the-art face recognition. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, pages 1528–1540. ACM, 2016.
- Huan Xu, Constantine Caramanis, and Shie Mannor. Robustness and regularization of support vector machines. *Journal of Machine Learning Research*, 10(Jul):1485–1510, 2009.